Procedimiento de implementación del software

Proyecto formativo · GPS Guardián Escolar

Equipo del proyecto GPS Guardián Escolar

5 de noviembre de 2026

Versión documental: 2.5.7 · **Estado:** versión editable preparada para subida y revisión de QA/arquitectura.

Tabla de contenido

Resumen y alcance operativo

Identificación del release y control de cambios

Arquitectura y secuencia de dependencias

Preparación del repositorio en Windows

Variables, usuarios y secretos

Respaldo y preflight

Despliegue Docker local / ambiente controlado

Migración Liquibase y roles

Implementación en Render (procedimiento de operador)

Verificación posterior y criterios de aceptación

Contingencia y rollback

Evidencias para cierre

Referencias y trazabilidad (APA 7.ª edición)

Corte y alcance de evidencia. Revisión documental 2.5.7, 5 de noviembre de 2026. La descripción del **release de referencia** utiliza los commits de main observados en Backend (1022058d), Frontend (3d9f59fa), DB (9278b1dc) e Infra (131e9895). Se inspeccionaron además las ramas dev/develop más recientes y los suplementos documentales 2.5.6, pero **sus cambios no se consideran integrados a producción por esa sola razón**. La revisión es estática: no se ejecutaron en esta sesión pruebas, despliegues ni validaciones en teléfono. Los resultados operativos requieren evidencia real.

Resumen y alcance operativo

Procedimiento controlado para preparar, desplegar, verificar y revertir GPS Guardian Escolar en un entorno Docker Compose de referencia y para documentar su promoción a Render. No se afirma que las órdenes se hayan ejecutado en esta revisión. Antes de modificar un ambiente, debe existir respaldo, aprobación y correspondencia entre la base de datos y el código desplegado.

1. Identificación del release y control de cambios

Registrar antes del despliegue los cuatro SHA aprobados; como referencia documental inicial se utilizaron: Backend 1022058d545a, Frontend 3d9f59fa3e91, Base 9278b1dcb224, Infra 131e98954430. No ejecutar automáticamente merges de dev/develop, qa, main, ya que se encontraron historias divergentes. Si las referencias cambiaron, actualizar la matriz y volver a validar.

Acta previa: entorno destino; responsable; fecha; cambio solicitado; riesgos; ticket/PR; aprobaciones QA; ventana; respaldo verificado; variables no secretas; plan de salida y rollback.

2. Arquitectura y secuencia de dependencias

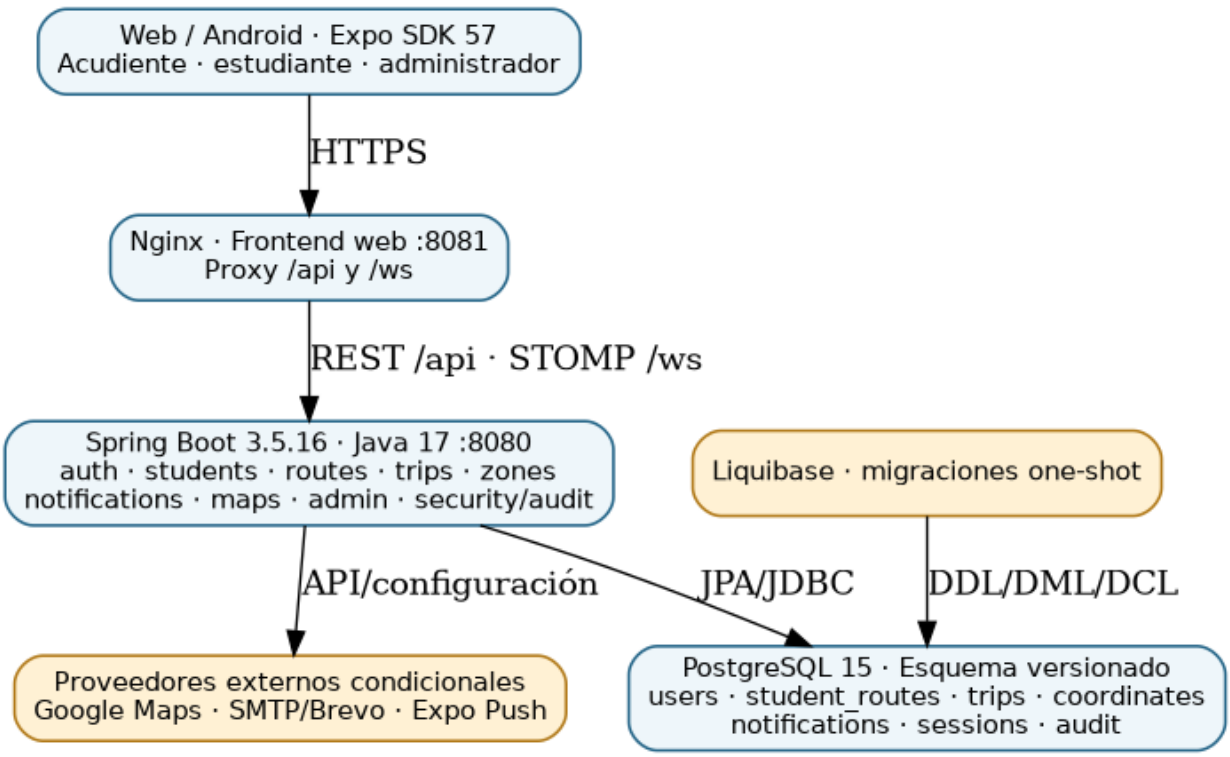


Figura 1. Topología del stack de referencia

database: PostgreSQL 15 arranca y supera pg_isready.

liquibase: aplica changesets una sola vez (service_completed_successfully).

backend: inicia con JDBC, credenciales restringidas y healthcheck /api/health.

frontend: Nginx sirve web y reenvía /api y /ws al backend.

Advertencia: en Render el servicio administrado y los contenedores pueden tener una topología distinta; la migración externa de PostgreSQL debe ejecutarse de forma planificada y **antes** de poner en línea la API que depende del nuevo esquema.

3. Preparación del repositorio en Windows

Con Git y Docker disponibles, colocar cuatro carpetas hermanas:

Guardian-Escolar/
  Backend-Guardian-Escolar/
  Fronted-GuardianEscolar-/
  DB-Structure-Guardian-Escolar/
  Docker-Infra-Guardian-Escolar/

En PowerShell: git clone por repositorio, git fetch --all --prune --tags, git switch/git checkout al SHA de release **en entorno descartable o de despliegue**, sin alterar la rama de trabajo del desarrollador. git status --short debe quedar sin cambios no registrados. Reportar hash en acta. Si hay commits nuevos de desarrollo no aprobados, no mover la referencia de release.

4. Variables, usuarios y secretos

Variable	Uso	Tratamiento
POSTGRES_PASSWORD	Bootstrap de DB y migración	Secreto exclusivo operador
APP_DB_USERNAME / APP_DB_PASSWORD	Rol DB de menor privilegio para API	Secreto separado del administrador; clave robusta
JWT_SECRET	Firma JWT	Aleatorio de al menos 32 bytes, privado
DB_URL, DB_USERNAME, DB_PASSWORD	JDBC backend directo	Privado y adaptado al entorno
APP_MAIL_ENABLED, MAIL_PROVIDER, MAIL_*, BREVO_API_KEY	Correo	No declarar entrega sin proveedor validado
GOOGLE_MAPS_API_KEY		

Proxy mapas servidor

Restringir/quitar de cliente cuando sea privada

EXPO_PUSH_ENABLED, EXPO_PUSH_URL

Notificaciones Expo

Registrar pruebas de recepción

EXPO_PUBLIC_*

Configuración compilada cliente

No introducir secretos

LIQUIBASE_CONTEXTS

Migraciones y bootstrap

Usar prod,bootstrap solo si aprobado; luego prod

En Infra, copiar .env.example a .env, completar contraseñas, JWT y rutas antes del arranque. No adjuntar .env real a GitHub, ZIP de evidencia ni capturas.

5. Respaldo y preflight

Antes de ejecutar migraciones en un entorno con datos, comprobar conectividad, tamaño y espacio; crear dump, cotejar salida y restaurar **en base de prueba**. Ejemplo genérico, ajustando credenciales y nombre de contenedor:

```
# Desde Docker-Infra-Guardian-Escolar, con .env privado válido
docker compose --env-file .env ps
docker compose --env-file .env exec -T database sh -c 'pg_dump -U "$POSTGRES_USER" -Fc "$POSTGRES_DB"' > .\backup_pre_release.dump
```

El archivo de respaldo contiene datos sensibles: guardarlo cifrado y con acceso restringido, nunca en el repositorio. Validar que sea legible y que se pueda restaurar en un ambiente aislado; un comando sin comprobación posterior no constituye respaldo certificado.

6. Despliegue Docker local / ambiente controlado

```
cd Docker-Infra-Guardian-Escolar
Copy-Item .env.example .env  # Solo si aún no existe; completar variables manualmente
docker compose --env-file .env config --quiet
docker compose --env-file .env up --build -d
docker compose --env-file .env ps
docker compose --env-file .env logs --tail=120 database liquibase backend frontend
```

Validar liquibase como exited (0) y los otros servicios como healthy cuando estén configurados. Usar Invoke-WebRequest http://localhost:8080/api/health y Invoke-WebRequest http://localhost:8081/. En versión publicada, probar las rutas por HTTPS desde otra máquina y el endpoint real de salud del release.

Atención a versiones actuales: el compose exige APP_DB_PASSWORD, incluso si algún manual anterior solo menciona POSTGRES_PASSWORD y JWT_SECRET. Sin ella, la configuración de la base de aplicación y el backend es incompleta.

7. Migración Liquibase y roles

La BD del producto está versionada en DB-Structure-Guardian-Escolar con DDL/DML/DCL y rollback. Revisar DATABASECHANGELOG y DATABASECHANGELOGLOCK; comprobar migración 014 de ruta preferida, constraints, roles y acceso del usuario gps_guardian_app sin privilegios de superusuario. Nunca editar changesets aplicados. El administrador inicial solo debe crearse/rotarse mediante contexto de bootstrap aprobado; retirar después las variables bootstrap de los servicios.

8. Implementación en Render (procedimiento de operador)

Verificar y registrar el servicio PostgreSQL y su URL externa/interna, TLS exigido, límites, almacenamiento y plan.

Preparar variables privadas del backend; comprobar DB_URL JDBC, rol restringido, JWT, orígenes CORS, puerto y proveedor de correo. No copiar contraseñas a documentación.

Ejecutar Liquibase contra el PostgreSQL de destino desde un job/host controlado con backup previo. Guardar registro de changesets y resultado; no suponer que Render ejecuta automáticamente migraciones.

Desplegar backend desde SHA aprobado y comprobar GET /api/health (o health endpoint efectivamente expuesto). Si falla, leer logs/Actuator sin exponer secretos.

Publicar frontend desde SHA aprobado. Configurar API_UPSTREAM hacia el backend, proxy /api y /ws y HTTPS. En web verificar rutas profundas Expo Router y WebSocket autenticado.

Ejecutar smoke tests de login, estudiante, ruta, viaje, coordenadas, notificación e historial con cuentas ficticias. Capturar fecha/hora, URL pública y SHA del despliegue.

Solicitar aprobación QA y registrar decisión de promoción. Sin evidencia real, el estado se mantiene «pendiente de certificación».

El despliegue previo en Render se recoge en documentación histórica del proyecto; esta guía no certifica la disponibilidad actual ni sustituye comprobar logs en el servicio correspondiente.

9. Verificación posterior y criterios de aceptación

Chequeo	Criterio y evidencia
Esquema	Liquibase terminó sin error y tablas/índices requeridos presentes
Salud	200 en API y respuesta HTTP del cliente; hora y URL
API	Login válido/inválido, rol ajeno rechazado y JWT revocado
Registro y GPS	Ruta preferida, viaje en curso, coordenada y STOMP bajo red real
Seguridad	No se exponen .env, tokens ni datos de menores
Externos	Maps/SMTP/Brevo/Push configurados y resultado observado
Backup	Dump cifrado y restauración de prueba trazable

10. Contingencia y rollback

Si el código falla pero el esquema es compatible, restaurar imágenes inmutables de frontend/backend anteriores con docker compose ... pull y up --no-build -d backend frontend. **No revertir SQL automáticamente:** las migraciones pueden haber modificado datos; hacer snapshot, estudiar dependencies y ejecutar rollback Liquibase revisado solo si es seguro. En incidente crítico, detener promoción, preservar logs y timestamps, considerar modo mantenimiento e informar al responsable.

No usar docker compose down -v para recuperar un servicio: **elimina el volumen de PostgreSQL**. El retorno debe registrarse con: incidente, operador, hash anterior/nuevo, resultado de salud, integridad de datos y aprobación final.

11. Evidencias para cierre

Adjuntar a un ticket privado: commits, PR, workflow CI asociado, salida de migración anonimizada, estado de contenedores/Render, capturas de UI reales, pruebas de contrato, aprobación QA y checklist de reversión. Ninguna evidencia fue fabricada o ejecutada durante la elaboración de esta edición.

Referencias y trazabilidad (APA 7.ª edición)

GPS Guardian Escolar. (2026a). *Backend-Guardian-Escolar* [Código fuente, rama main, commit 1022058d]. GitHub. https://github.com/GPS-Guardian-Escolar/Backend-Guardian-Escolar/tree/1022058d545a9e7936edbff75950034ec4f98e88

GPS Guardian Escolar. (2026b). *Fronted-GuardianEscolar-* [Código fuente, rama main, commit 3d9f59fa]. GitHub. https://github.com/GPS-Guardian-Escolar/Fronted-GuardianEscolar-/tree/3d9f59fa3e91b5ecc5b8ae86591187546b966668

GPS Guardian Escolar. (2026c). *DB-Structure-Guardian-Escolar* [Migraciones SQL, rama main, commit 9278b1dc]. GitHub. https://github.com/GPS-Guardian-Escolar/DB-Structure-Guardian-Escolar/tree/9278b1dcb224301536d6158357f74daee74b20ba

GPS Guardian Escolar. (2026d). *Docker-Infra-Guardian-Escolar* [Configuración de despliegue, rama main, commit 131e9895]. GitHub. https://github.com/GPS-Guardian-Escolar/Docker-Infra-Guardian-Escolar/tree/131e98954430e95e9108174eded9896b3f0d5272

GPS Guardian Escolar. (2026e). *Docs-Guardian-Escolar: entregables 2.5.5 y suplementos 2.5.6* [Documentación del proyecto]. GitHub. https://github.com/GPS-Guardian-Escolar/Docs-Guardian-Escolar

Fuentes primarias relevantes: TripController.java, TripService.java, StudentController.java, LocationAvailabilityService.java, WebSocketConfig.java, application.yml, package.json, docker-compose.yaml y 01_ddl/04_alter/014_preferred_student_route.sql, según el documento. Las referencias a rutas son comprobables en los repositorios indicados.